hw1

Contents

Part A: Code Implementation

Functions

```r
# Problem 1 ----------------------------------------------------------------------------

logdet <- function(R) {
    #' `logdet` function calculates the log of the determinant of the matrix.
    #'
    #'  Args:
    #'  ----
    #'  - R (matrix): the target matrix
    #'
    #'  Returns:
    #'  --------
    #'  - output (num): the log of the det of R

    eigs <- eigen(R)$values

    if (any(eigs <= 0)) {
        # handle non-positive eigenvalues case
        stop("matrix R is not positive definite")
        return(NA_real_)
    }
    # det(R) = the product of eigen-values of a matrix
    res <- sum(log(eigs))

    return(res)
}


# Problem 2 ----------------------------------------------------------------------------
```

```r
logmarglik <- function(data, A) {
    #' `logmarglik` calculates the marginal log likelihood given the data and features.
    #'
    #' Args:
    #' -----
    #' - data (matrix | data.frame-like): (n x p) matrix-like data
    #' - A (vector): vector selecting target columns of data
    #'
    #' Return:
    #' -------
    #' - logmarglik (num): the log likelihood value

    m <- as.matrix(data) # NOTE: to accomodate both data.table and data.frame
    n <- nrow(m)
    k <- length(A)


    D1 <- m[, 1]
    DA <- m[, A] # NOTE: data.table will fail with this syntax

    MA <- diag(k) + t(DA) %*% DA
    temp <- (1 + t(D1) %*% D1 - t(D1) %*% DA %*% solve(MA) %*% t(DA) %*% D1)
    loglik <- lgamma((n+k+2)/2) - lgamma((k+2)/2) - 0.5*logdet(MA) - ((n+k+2)/2) * log(temp)

    return(as.numeric(loglik))
}
```

Data

```r
df <- read.csv("hw1/erdata.txt", sep = "\t", header = F)
print(dim(df))
```

```
## [1] 158  51
```

```r
loglik <- logmarglik(df, A = c(2, 5, 10))
print(loglik)
```

```
## [1] -59.97893
```

Part B: AI solution

- Model: Gemini 3 - Fast

Prompt

```r
# Finish the following 2 tasks with R language:
#
# 1. Write a function in R that computes the logarithm of the determinant of a matrix R. The
# function call should be `logdet(R)`, where R is a matrix-like object (could take data.frame).
# You can exploit the fact that the determinant of a matrix is given by the product of the
# eigenvalues of the matrix. You can use the function "eigen" to find these eigenvalues.
# Also, please handle the case that the R matrix is not positive definite to prevent errors.
#
```

```
# here is the function designa pattern:
# logdet <- function(R) {
#     # 1. handle R input to be matrix
#     # 2. main: calc. logdet
#     # 3. return the log det value
# }
#
#
# 2. Write a function in R that computes the logarithm of the marginal likelihood
# associated with the regression of the response on a subset of explanatory variables A
# {2,...,p}. The function call should be: `logmarglik(data, A)`.
# This assumes that "data" represents the n×p data matrix (could also be data.frame|data.table).
# The marginal likelihood is defined by formula below.
# You essentially need to calculate log(p(D1,DA|[1|A])).
# The marginla likelihood formula: p(D1,DA|[1|A])
# = gamma((n+|A|+2)/2) / gamma((|A|+2)/2) * (det(MA))^{-1/2} *
# (1 + D1^t%*%D1 - D1^t %*% DA %*% MA^{-1} %*% DA^t %*% D1)^{(n+|A|+2)/2},
# where |A| denotes the number of elements of A, D1 is the response vector,
# DA is the feature subset matrix (i.e., data[, A]).
# More context: P(D1|DA, \beta_A) ~ N(\sum_j \beta_j x_j, \sigma^2).
# where P(\sigma^2 | D1, DA) ~
# InverseGamma((n + |A| + 2)/2, (1 + D1^t%*%D1 - D1^t %*% DA %*% MA^{-1} %*% DA^t %*% D1));
# and P(\beta_A | \sigma^2, D1, DA) ~ MultiNormal(MA^{-1}%*%DA^t%*%D1, \sigma^2*MA^{-1});
# MA = I + DA^t %*% DA.
# The D^t, means the transpose of the D matrix.
#
# here is the design pattern:
# logmarglik <- function(data, A) {
#     # 1. handle data format, accepting data.frame and matrix-like object
#     # 2. main: calc. the marginal log-likelihood with logdet() func.
#     # 3. output the loglik value
# }
#
# Design these 2 functions with proper docstrings.
```

Response

Problem 1

Here are the R functions designed according to your specifications. I have utilized the eigen approach for the determinant and implemented the marginal likelihood using logarithmic transformations to ensure numerical stability.1. Log-Determinant FunctionThis function calculates $\log(\det(R))$ using the identity that the determinant of a matrix is the product of its eigenvalues: $\det(R) = \prod \lambda_i$, which implies $\log(\det(R)) = \sum \log(\lambda_i)$.

```
#' Compute the logarithm of the determinant of a matrix
#'
#' @param R A matrix-like object (matrix, data.frame, etc.)
#' @return The log-determinant value. Returns NA if the matrix is not positive definite.
logdet <- function(R) {
  # 1. Handle R input to be matrix
  R_mat <- as.matrix(R)

  # 2. Main: calc. logdet using eigenvalues
```

```
  # Determinant = product of eigenvalues; log(det) = sum(log(eigenvalues))
  ev <- eigen(R_mat, symmetric = TRUE, only.values = TRUE)$values

  # 3. Handle non-positive definite cases (eigenvalues must be > 0 for log)
  if (any(ev <= 0)) {
    warning("Matrix is not positive definite. Returning NA.")
    return(NA)
  }

  log_det_val <- sum(log(ev))

  # 4. Return the log det value
  return(log_det_val)
}
```

Problem 2

2. Log-Marginal Likelihood FunctionThis function implements the provided marginal likelihood formula. To prevent numerical overflow/underflow, we use lgamma and the logdet function defined above.The formula is computed as:

$$\log(p) = \log\Gamma\left(\frac{n+|A|+2}{2}\right) - \log\Gamma\left(\frac{|A|+2}{2}\right) - \frac{1}{2}\log(\det(M_A)) - \frac{n+|A|+2}{2}\log(1+D_1^T D_1 - D_1^T D_A M_A^{-1} D_A^T D_1)$$

```
#' Compute the logarithm of the marginal likelihood
#'
#' @param data An n x p data matrix or data.frame.
#' @param A A vector of indices representing the subset of explanatory variables.
#' @return The log marginal likelihood value.
logmarglik <- function(data, A) {
  # 1. Handle data format
  data_mat <- as.matrix(data)
  n <- nrow(data_mat)
  size_A <- length(A)

  # Extract components
  D1 <- data_mat[, 1, drop = FALSE]       # Response vector (first column)
  DA <- data_mat[, A, drop = FALSE]       # Feature subset

  # Calculate MA = I + DA^T %*% DA
  # diag(size_A) creates an Identity matrix of size |A|
  MA <- diag(size_A) + t(DA) %*% DA

  # 2. Main: calc. the marginal log-likelihood

  # Part A: Log Gamma terms
  term_gamma <- lgamma((n + size_A + 2) / 2) - lgamma((size_A + 2) / 2)

  # Part B: -1/2 * log(det(MA))
  term_det <- -0.5 * logdet(MA)

  # Part C: The quadratic form component
  # We use solve(MA) to get the inverse. For efficiency: DA %*% solve(MA, t(DA))
  # Quad = 1 + D1'D1 - D1'DA(MA^-1)DA'D1
  quad_core <- t(D1) %*% DA %*% solve(MA) %*% t(DA) %*% D1
```

```
  quad_term <- 1 + as.numeric(t(D1) %*% D1) - as.numeric(quad_core)

  # Final sum in log space
  # Note: The formula uses (1 + ...)^-(power), which becomes -power * log(1 + ...)
  log_ml <- term_gamma + term_det - ((n + size_A + 2) / 2) * log(quad_term)

  # 3. Output the loglik value
  return(as.numeric(log_ml))
}
```

Empirical Result

```
logmarglik(df, A = c(2, 5, 10))
```

```
## [1] -59.97893
```

Part C: Solutions Comparison

I asses the solutions in the following aspects.

Overall, the logics and the outputs are the same between my and AI's solution given the input.

Regarding `logdet` function, AI's solution is more efficient under stronger assumption on input matrix, aslo handles the non-PD matrix case better. As for `logmarglik`, AI's solution is a bit too verbose, but its calculation would be more clear when taking long equations into smaller terms.

Core Logic

- The core logic are from the same formulas, the code implement logics are basically the same.
- The AI solution outputs the same log-likelihood value as my implementation

Efficiency/Scalability

- AI solution will perform slightly better/faster when calculating the determinant of matrix, since it set symmetric option to true, given its knowledge to the formation of our R matrix; My solution did not leverage this fact of our R input, making the calculation slower when facing larger D_A matrix.
- However, I think the `logdet()` function could be more general and takes less assumption, since it is not originally designed "only" for regression feature matrix case.
- But it's still useful to know that there is a symmetric option in `eigen` function to accelerate the calculation.

Stability

- My solution check the case for R being non-positive definite matrix, which leads to error for `logdet` function. The function will stop when encountering a non-PD input.
- The AI solution handles this case using warning and outputs NA, which is more practical in production in my opinion.
- Both my and AI's solutions convert the input data/matrix first in the functions, making the design more robust under different input data types.

Coding Style/Readability

- Generally, AI's solution is more verbose with comments. Also, it takes the calculation into more parts that could help the readability. But some of the comments are too redundant (e.g. -1/2 * log(det(MA))), it basically comments what it had written.